Année scolaire 2026-2027

Lycée Victor Hugo - Colomiers

Première NSI

LP
Langages de Programmation

Thomas Ricart

ricart.lvh@laposte.net

https://thomasricart.github.io

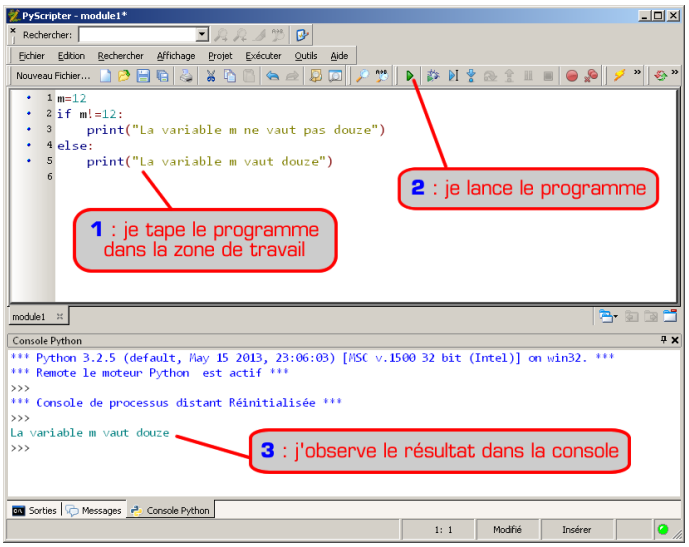
27 mars 2026

LP01 CONSTRUCTIONS ÉLÉMENTAIRES

I. L'environnement de travail

Nous avons à notre disposition plusieurs outils pour écrire du code Python.

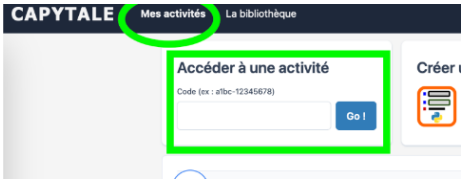
Edupython



`Edupython` étant installé localement, il se peut que des obglets soient déjà ouverts. Fermez tous les onglets ouverts et ouvrez un nouvel onglet pour commencer à travailler.

Notebook Capytale

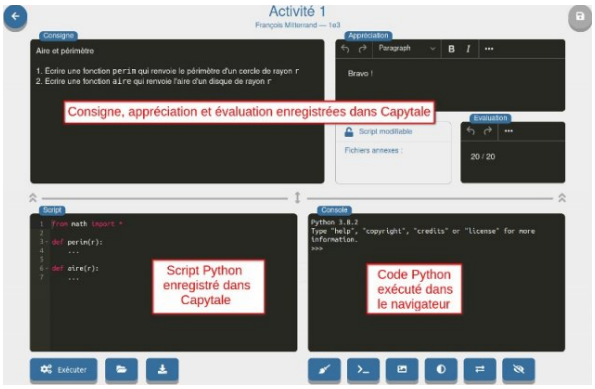
Vous accédez à `Capytale - Notebook` depuis votre ENT en cliquant sur `Pédagogie/Capytale`. Le professeur vous donnera un code à entrer dans la zone `Accéder à une activité`.



Vous pourrez également y accéder directement via un lien que vous donnera votre enseignant.

Console Capytale

Vous accédez à `Capytale - Console` depuis votre ENT de la même manière que pour les Notebook.



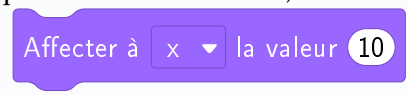
Quelque soit la façon dont vous tapez du code, n'oubliez pas de sauvegarder TRES régulièrement.

II. Les variables en Python

Déclarer une variable en Python

Pour déclarer une variable en Python, il suffit de lui donner un nom et de lui affecter une valeur à l'aide du symbole =. Par exemple, pour déclarer une variable `x` qui contient la valeur 10, on écrit :

Code Python
```
a = 10
```



Le nom des variables en Python

- Elles doivent commencer par une lettre ou un underscore (_).
- Elles peuvent contenir des lettres, des chiffres et des underscores.
- Elles ne peuvent pas être des mots réservés du langage Python (comme `if`, `for`, `while`, etc.).
- `Variable` et `variable` sont considérées comme deux variables différentes.

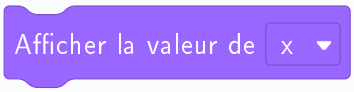
Exemple : Noms de variables valides et invalides

Bonne variable	`ma_variable`
Mauvaise variable	`Ma_variable`, `ma variable`, `1_variable`, `variable@`
A éviter	`Variable non explicites`

Afficher une variable en Python

Pour afficher la valeur d'une variable en Python, on utilise la fonction `print()`. Par exemple, pour afficher la valeur de la variable `x`, on écrit :

Code Python
```
print(x)
```



Les types de variables en Python

- **int** : pour les nombres entiers (ex : 1, 42, -5)
- **float** : pour les nombres à virgule flottante (ex : 3.14, -0.001)
- **str** : pour les chaînes de caractères (ex : "Bonjour", 'Python')
- **bool** : pour les valeurs booléennes (ex : True, False)

```
# type (int)
a = 5
b = 6
```

```
# type (str)
a = 'Bonjour'
b = '6'
```

```
# type (float)
a = 5.0
b = 6.8
```

```
# type (bool)
a = True
b = False
```

Opérations mathématiques de base en Python

- `+` addition : `5 + 3` donne 8
- `-` soustraction `5 - 3` donne 2
- `*` multiplication `5 * 3` donne 15
- `/` division `6 / 2` donne 3.0 (`float`)
- `**` puissance `2 ** 3` donne 8
- `%` reste de la division entière `7 % 3` donne 1
- `//` quotient de la division entière `7 // 3` donne 2

```
>>> print(17+2)
19
>>> print(17-2)
15
```

```
>>> print(17*2)
34
>>> print(17/2)
8.5
```

```
>>> print(17%2)
1
>>> print(17//2)
8
```

Exercice 1 : Repérer les les types de variables et leurs valeurs dans les programmes suivants

- `x = 5`
- `y = "Hello"`
- `z = True`
- `a = 3.14`
- `c = 6/2`
- `d = 6//2`
- `f = '1' + '2'`
- `g = 2**3`

III. Les variables de type chaine de caractère `str`

Les chaines de caractères

Les chaines de caractères sont des variables qui contiennent du texte. Elles sont délimitées par des guillemets simples ou doubles.

```
>>> a = 'Bonjour'
>>> b = ' a tous'
>>> c = a + b
>>> print(c)
Bonjour a tous
```

```
>>> a = 'Bonjour'
>>> b = ' a tous'
>>> c = a * 3
>>> print(c)
BonjourBonjourBonjour
```

```
>>> a = 'Bonjour'
>>> b = ' a tous'
>>> c = a[0]
>>> print(c)
B
```

Dans l'exemple ci-dessus, la variable `c` contient la concaténation de `a` et `b` dans le premier cas, la répétition de `a` trois fois dans le deuxième cas et le premier caractère de `a` dans le troisième cas.

indexation des chaines de caractères

Les chaines de caractères peuvent être vues comme un ensemble de caractères "collés" les uns aux autres Chaque caractère d'une chaine de caractères est indexé : il y a un index qui donne sa position dans la chaine

Indice	0	1	2	3	4	5	6
Caractère	B	o	n	j	o	u	r
Indice Négatif	-7	-6	-5	-4	-3	-2	-1

```
>>> mot = 'Bonjour'
>>> mot[0]
'B'
>>> mot[4]
'o'
```

```
>>> mot = 'Bonjour'
>>> n = len(mot)
>>> n
7
>>> mot[-1]
'r'
```

Les opérations sur les chaines de caractères

- `+` pour la concaténation : `'Bonjour' + ' a tous'` donne `'Bonjour a tous'`
- `*` pour la répétition : `'Bonjour' * 3` donne `'BonjourBonjourBonjour'`
- `[]` pour l'indexation : `'Bonjour'[0]` donne `'B'`
- `in` pour le test d'appartenance : `'on' in 'Bonjour'` donne `True`
- `not in` pour le test d'appartenance : `'on' not in 'Bonjour'` donne `False`
- `len()` pour la longueur : `len('Bonjour')` donne `7`

```
>>> mot1 = 'Bonjour'
>>> mot2 = 'Thomas'
>>> mot1 + ' ' + mot2
'Bonjour Thomas'
```

```
>>> 'j' in 'Bonjour'
True
>>> 'b' in 'Bonjour'
False
```

```
>>> 'abracadabra'[4]
'c'
>>> len('abracadabra')
11
```

Opérations avancées sur les chaines de caractères : le Slicing

`[a:b]`	conserve les caractères d'indice `[a:b]` : `'Bonjour'[1:4]` donne `'onj'`
`[a:]`	conserve les caractères d'indice `[a:]` : `'Bonjour'[3:]` donne `'jour'`
`[:b]`	conserve les caractères d'indice `[:b]` : `'Bonjour'[:3]` donne `'Bon'`
`[::-1]`	inverse les caractères : `'Bonjour'[::-1]` donne `'ruojnoB'`
`[1:-1]`	élimine le premier et dernier caractère : `'Bonjour'[1:-1]` donne `'onjou'`

Exercice 2 : Prédire ce que vont afficher les codes suivants

```
>>> mot = 'Numerique'
>>> n = len(mot)
>>> mot[5]
>>> mot[n]
>>> mot[-1]
>>> mot[8]
```

```
>>> mot = 'Numerique'
>>> m in mot
>>> e not in mot
>>> mot+mot[2]
>>> mot*2
>>> 'abc'+mot
```

```
>>> mot = 'Numerique'
>>> mot[1:5]
>>> mot[:4]
>>> mot[3:]
>>> mot[1:-1]
```

Principe du transtypage

Les types des variables en python sont dynamiques, on peut modifier sa valeur ainsi que son type.

```
>>> a = '5'      # a est de type str
>>> b = int(a)   # b est de type int
>>> print(b)
5
```

```
>>> a = 4        # a est de type int
>>> b = str(a)   # b est de type str
>>> print(b)
4
```

IV. Les variables de type booléen `bool`

Les booléens

Les booléens sont des variables qui ne peuvent prendre que deux valeurs : `True` (vrai) ou `False` (faux).

Les booléens peuvent être utilisés dans des opérations logiques.

```
>>> a = True
>>> b = False
>>> c = a and b
>>> c
False
```

```
>>> a = True
>>> b = False
>>> c = a or b
>>> c
True
```

```
>>> a = True
>>> c = not a
>>> c
False
```

Table de vérité du `not`

Expression	Résultat
`not` True	False
`not` False	True

Table de vérité du `and`

Expression	Résultat
True `and` True	True
True `and` False	False
False `and` True	False
False `and` False	False

Table de vérité du `or`

Expression	Résultat
True `or` True	True
True `or` False	True
False `or` True	True
False `or` False	False

Booléen comme résultat de comparaison

Les booléens sont le résultat d'une comparaison. Les opérateurs de comparaison sont :

- `==` pour le test d'égalité
- `!=` pour le test de différence
- `<` pour le test d'infériorité
- `>` pour le test de supériorité
- `<=` pour le test d'infériorité ou d'égalité
- `>=` pour le test de supériorité ou d'égalité

Booléen comme résultat de comparaison

```
>>> a = 5
>>> b = 6
>>> c = (a == b)
>>> c
False
```

```
>>> a = 5
>>> b = 6
>>> c = (a != b)
>>> c
True
```

```
>>> a = 5
>>> b = 6
>>> c = (a < b)
>>> c
True
```

Exercice 3

Quelle est la valeur de la variable `c` après l'exécution des codes suivants ?

```
>>> a = 3
>>> b = 1
>>> c = a > b
```

```
>>> a, b = True, False
>>> c = (a and b) or (not a)
```

```
>>> a = True
>>> b = False
>>> c = (a or b) and (not a)
```

```
>>> a = True
>>> b = False
>>> x, y = 7, 9
>>> c = (b or (x == y)) or (a and (x != y))
```

LP02 LES FONCTIONS

I. La gestion des entrées - sorties

Gestion des entrées en Python

Il est possible en python de demander à l'utilisateur d'entrer une valeur au clavier. Cela s'effectue via la commande `input` La valeur tapée par l'utilisateur est convertie au format chaine de caractère (str)

Code Python

```python
a = input("Entrer une valeur au clavier")          # a sera de type str
b = int(input("Entrer une valeur au clavier"))     # b sera de type int
```

Méthode 1 : Affichage dans la zone programme

Code Python

```python
def f(x):
    y = x ** 2  # Fonction calculant x**2
    return y

f(4)            # PAS D'AFFICHAGE: f(4) est créé mais est perdue
print(f(4))     # AFFICHAGE: f(4) créé puis affiché. La valeur est perdue
carre = f(6)    # PAS D'AFFICHAGE: f(6) créé puis sauvegardé dans carre.
print(carre)    # AFFICHAGE: affichage de la valeur de la variable carre
```

On peut afficher des chaines complexes avec le séparateur virgule ,

Code Python

```python
a, b = 6, 4
print("La somme de", a, "et",b, "est", a + b)
```

Méthode 2 : Affichage dans la console python

La console python accepte en plus de la commande `print` un affichage direct en appelant la variable.

```
>>> a = 6
>>> a
6
```

Méthode 3 : Formatage de l'affichage avec `f-string` et `format()`

L'idée est de créer un texte à trou que l'on va compléter par des variables.

```
>>> name = 'Paul'
>>> age = 25
>>> print(f'Votre nom est {name} et vous avez {age} ans.')
Votre nom est Paul et vous avez 25 ans.
```

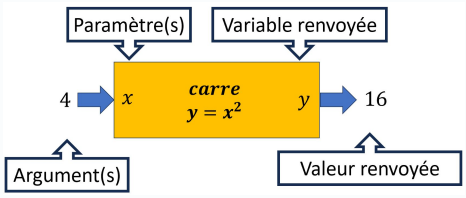
```
>>> print('Votre nom est {} et vous avez {} ans'.format(name, age))
Votre nom est Paul et vous avez 25 ans
```

II. Les fonctions

Présentation

Une **fonction** en python est un objet qui se comporterait comme une fonction mathématique. On peut prendre pour exemple une fonction f définie par $f(x) = x^2$.

- f est le nom de la fonction
- x est un paramètre (une variable en mathématique). C'est juste un concept dont on va se servir pour appliquer la fonction. Ce nom de variable n'est connu qu'à l'intérieur de la fonction.
- x^2 est l'expression de la fonction. C'est la manière dont on va calculer l'image de x par la fonction f.



- Ici la valeur 4 est un argument, c'est à dire la valeur sur laquelle on va appliquer la fonction.
- La valeur 16 est la valeur renvoyée par la fonction. Ne pas confondre `variable renvoyée` et `valeur renvoyée`

Code Python

```
1 def f(x):
2     """ Fonction renvoyant x ** 2 """
3     y = x ** 2          # Calcul de x ** 2
4     return y            # valeur renvoyée: y
```

- **Ligne 1 :** `def` permet de définir le nom de la fonction. Ici la fonction se nomme `f`.
- **Ligne 1 :** `f(x)` permet de définir le nom et le nombre des paramètres. Ici la fonction `f` a besoin d'un seul paramètre `x`
- **Ligne 2 :** commentaire permettant de décrire ce que fait la fonction. Cette ligne n'est pas interprétée
- **Ligne 3 :** on lit de la droite vers la gauche : « on prend le contenu de la variable `x`, on le met au carré et on l'affecte à une nouvelle variable appelée `y` »
- **Ligne 4 :** on renvoie la valeur de la variable `y`.

On vient de définir une nouvelle fonction. Il faut maintenant lui donner un argument pour qu'elle puisse calculer une valeur et la renvoyer. C'est ce que l'on appelle **appeler** la fonction.

Code Python

```
a = 4           # on affecte 4 à la variable a
z = f(4)        # on appelle la fonction f avec comme argument 4
                # et on récupère la valeur renvoyée dans la variable z
```

- la commande `return` renvoie la valeur contenue dans y
- elle provoque la sortie immédiate de la fonction
- si possible on ne met qu'un seul `return` dans la fonction
- il peut n'y avoir aucun `return`, on parle alors de `procédure`

Prototypage d'une fonction

Code Python

```
def ma_fonction(param1, param2, ...):
    ''' Docstring: Explication de ce que fait ma_fonction
    : description des paramètres et leurs types
    : description des valeurs de sortie et leurs types '''
    # Corps de la fonction

    return valeur     # La variable 'valeur' est renvoyée
```

Exemple

Voici un exemple d'une fonction qui fait quelque chose de très simple : elle prend deux valeurs en paramètres (`a` et `b`) et renvoie la somme des ces deux valeurs. Cette somme a été créée et affectée à la variable `somme`.

Code Python
```python
def calc_somme(a, b):
    ''' Fonction calculant la somme de deux valeurs
    : a et b deux entiers (int)
    : valeur renvoyée est un entier (int) '''
    somme = a + b        # le résultats de a + b est affecté à la variable somme
    return somme         # la valeur de somme est renvoyée par la fonction
```

Il est maintenant possible de se servir de cette fonction.

Code Python
```python
s = calc_somme(6, 8)     # s récupère la valeur renvoyée par calc_somme(6, 8)
print(s)                 # la valeur de la variable s est affichée (14)
```

Remarque : Variables locales et variables globales

- les **variables locales** ne sont connues que dans la fonction
- les **variables globales** sont connues dans tout le programme

Code Python
```python
print(a)    # ERREUR -> a est une variable locale (tout comme b et somme)
```

Il est donc tout à fait normal que les variables `a`, `b` et `somme` ne soient pas connues à l'extérieur de la fonction. Il est de bon usage de ne pas appeler de la même façon des variables locales et des variables globales.

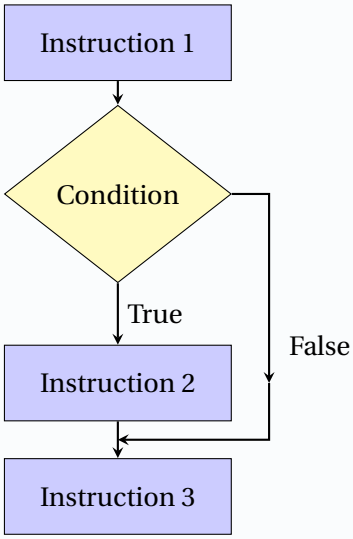
Vérification de la valeur d'une variable

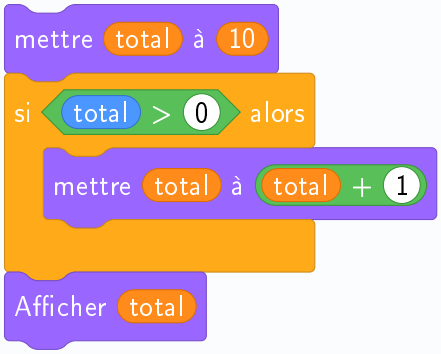
La fonctionnalité `assert` vérifie qu'une affirmation est vraie. Si c'est le cas, rien ne se passe, sinon un message d'erreur est donné

Code Python
```python
assert calcule_somme(6, 8) == 14          # True -> aucune message
assert calcule_somme(6, 8) == 12          # False -> message d'erreur
assert not(calcule_somme(6, 8) == 12)     # True -> aucun message
```

LP03 CONDITIONS IF

Les structures conditionnelles IF






```python
# En python
Instruction_1
if condition:
    Instruction_2
Instruction_3
```

```python
# En python
total = 10
if total > 0:
    total = total + 1
print(total)
```

- l'`Instruction 1` est effectuée
- SI la `Condition` est `True` alors l'`Instruction 2` est effectuée
- SINON on va directement à l'`Instruction 3`
- Dans tous les cas l'`Instruction 3` est effectuée

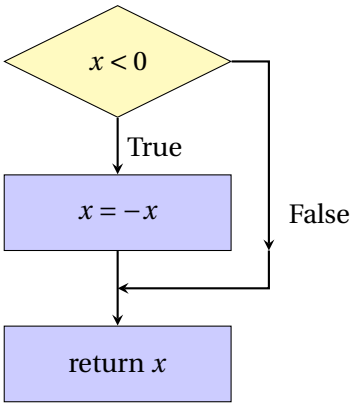
La `Condition` est un test qui est évalué en python par un booléen.

Exemple : La condition IF sans ELSE : la valeur absolue d'un nombre

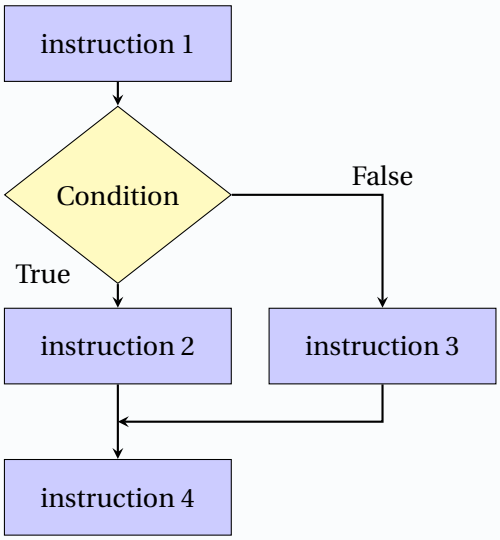
- si $x \geqslant 0$, alors $|x| = x$
- si $x < 0$, alors $|x| = -x$

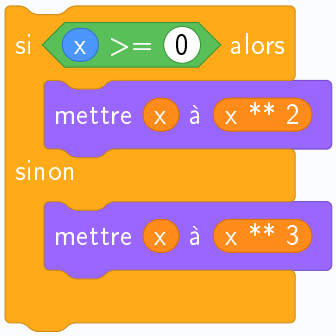
Code Python

```python
def valeur_absolue(x):
    ''' Renvoie |x| '''
    if x < 0:
    # si x < 0 on échange x par -x
        x = - x
    return x
```



Les structures conditionnelles IF – ELSE






```python
# En python

instruction_1

if condition:
    instruction_2
else:
    instruction_3
instruction_4
```

- l'`instruction 1` est effectuée
- SI la `Condition` est `True` alors l'`instruction 2` est effectuée
- SINON l'`instruction 3` est effectuée
- Dans tous les cas l'`instruction 4` est effectuée

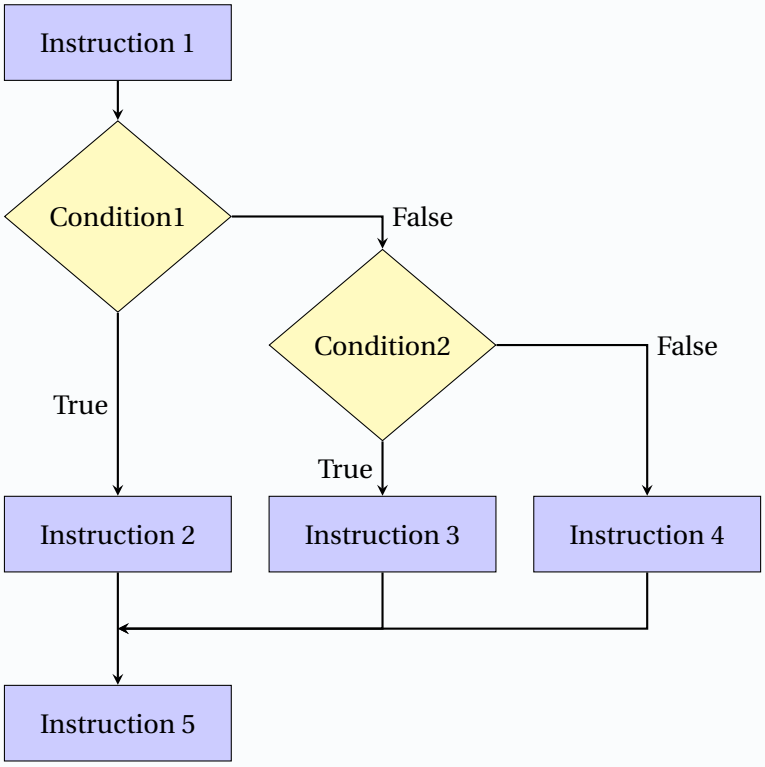
Exemple : La condition IF – ELSE

Code Python
```python
def ma_fonction(x):
    if x >= 0:
        sortie = x ** 2
    else:
        sortie = x ** 3
    return sortie
```

Les structures conditionnelles IF – ELIF – ELSE

Dans le cas où il y a plusieurs possibilités de réponse, on peut utiliser deux structures différentes en python. On suppose une fonction prenant comme paramètre une variable x de type entier.

$$f(x) = 1 \text{ si } x > 0 \qquad f(x) = 0 \text{ si } x = 0 \qquad f(x) = -1 \text{ si } x < 0$$



Code Python
```python
def ma_fonction(x):
    if x > 0:
        sortie = 1
    elif x == 0:
        sortie = 0
    else:
        sortie = -1
    return sortie
```

Code Python
```python
def ma_fonction(x):
    if x > 0:
        sortie = 1
    else:
        if x == 0:
            sortie = 0
        else:
            sortie = -1
    return sortie
```

Exercice 1 : Compléter les fonctions suivantes

Code Python
```python
def plus_grand2(a, b):
    ''' Renvoie le plus grand des
    deux nombres a et b '''
    # A compléter



    #
```

Code Python
```python
def plus_grand3(a, b, c):
    ''' Renvoie le plus grand des
    trois nombres a, b et c '''
    # A compléter



    #
```

LP04 BOUCLES FOR ET WHILE

I. La boucle FOR

Les boucles FOR

La boucle `FOR` est une boucle bornée. On sait à l'avance combien de fois on va la faire.

```
# En pseudo code
instruction1
Pour i allant de 0 a 6:
    instruction2
instruction3
```

- instruction1 est effectuée
- `i` est un `itérateur` qui va aller de 0 à 6, c'est à dire 7 tours
- On effectuera donc 7 fois l'instruction2
- Quand on a fini la boucle on effectue l'instruction3

La boucle FOR en python

Code Python
```python
for i in range(0, 7):
    print(i, end=" ")
# affichage:
# 0 1 2 3 4 5 6
```

Code Python
```python
for i in range(2, 7):
    print(i, end=" ")
# affichage:
# 2 3 4 5 6
```

Code Python
```python
for i in range(5):
    print(i, end=" ")
# affichage:
# 0 1 2 3 4
```

Prototypage de la boucle FOR

La boucle FOR s'écrit de manière générale de la façon suivante :

Code Python
```python
for i in range(a, b):
```

La boucle est effectuée $b-a$ fois et la variable `i` prend pour valeur tous les entiers entre $[a;b[$

Exemple : Trace de la structure FOR

Code Python
```python
for i in range(2, 6):
    print(i)
print('fin')
```

i	i < 6	instruction
2	True	print(2)
3	True	print(3)
4	True	print(4)
5	True	print(5)
6	False	print('fin')

Itération sur une chaine de caractères via les indices

Une chaine de caractères `str` est un itérateur, c'est à dire qu'il est possible d'itérer sur ses éléments.

```
>>> mot = 'Hello'
>>> mot[1]
'e'
```

Indice	0	1	2	3	4
Caractère	H	e	l	l	o
Indice Négatif	-5	-4	-3	-2	-1

Code Python
```python
mot = 'Hello'
for i in range(0, len(mot)):
    print(mot[i])
print('Fin')
```

i	i < len(mot)	instruction	affichage
0	True	print(mot[0])	H
1	True	print(mot[1])	e
2	True	print(mot[2])	l
3	True	print(mot[3])	l
4	True	print(mot[4])	o
5	False	print('Fin')	Fin

Itération sur une chaine de caractères via les éléments

Il est aussi possible d'itérer directement sur les éléments qui constituent cette chaine de caractère
On peut donc créer un variable nommée `lettre` par exemple qui prendra pour valeur chacun des caractères de `mot`.

Code Python

```python
mot = 'Hello'
for lettre in mot:
    print(lettre)
print('Fin')
```

lettre	instruction	affichage
'H'	print(lettre)	H
'e'	print(lettre)	e
'l'	print(lettre)	l
'l'	print(lettre)	l
'o'	print(lettre)	o
None	print('Fin')	Fin

- Il n'y a pas besoin de connaitre la longueur du mot
- Il n'y plus l'utilisation d'indice
- `lettre` prend pour valeur chaque caractère de mot
- lorsque `lettre` a pris pour valeur tous les caractères de `mot`, la boucle s'arrête.

Exercice 1

Écrire une fonction permettant d'afficher tous les entiers entre 0 et 6 inclus

Exercice 2

Écrire une fonction permettant d'afficher tous les entiers entre 2 et 9 inclus

Exercice 3

Écrire une fonction permettant d'afficher toutes les lettres du mot `'abracadabra'` en itérant sur les indices

Exercice 4

Écrire une fonction permettant d'afficher toutes les lettres du mot `'abracadabra'` en itérant sur les caractères

Algorithmes de Recherche dans un itérable

Cet algorithme permet de savoir si un élément est présent dans un itérable.

- Le mot sera la variable `mot`
- La lettre que l'on cherche sera la variable `lettre_cherchee`

On peut utiliser l'algorithme suivant :

1. on parcourt `mot` à l'aide d'une boucle `FOR`
2. on observe donc chaque caractère `mot[i]`
3. si le caractère est égal à `lettre_cherchee`, alors on renvoie la valeur `True` et on arrête la boucle.
4. sinon on ne fait rien et on revient à l'étape 2 pour observer le caractère suivant.
5. si on a observé tous les caractères de `mot` sans trouver `lettre_cherchee`, on renvoie `False`

Exercice 5 : Écrire une fonction `est_present(mot, lettre_cherchee)`

Code Python

```python
def est_present(mot, lettre_cherchee):
    ''' Renvoie True si lettre_cherchee est dans mot et False sinon'''




    #
```

Exercice 6

Ecrire la fonction `est_present2(mot, lettre_cherchee)` en itérant sur les indices

Exercice 7

Ecrire la fonction `est_present3(mot, lettre_cherchee)` qui renvoie l'indice de la première occurrence de lettre_cherchee dans mot ou `-1` si elle n'est pas présente

Algorithmes de comptage dans un itérable

Cet algorithme permet de compter le nombre de fois qu'un élément est présent dans un itérable.

- Le mot sera la variable `mot`
- La lettre que l'on cherche sera la variable `lettre_cherchee`

On peut utiliser l'algorithme suivant :

1. on initialise un compteur à 0
2. on parcourt `mot` à l'aide d'une boucle `FOR`
3. on observe donc chaque caractère `mot[i]`
4. si le caractère est égal à `lettre_cherchee`, alors on incrémente le compteur.
5. sinon on ne fait rien
6. on revient à l'étape 3 pour observer le caractère suivant.
7. si on a observé tous les caractères de `mot`, on renvoie le compteur

Exercice 8 : Écrire une fonction `compte(mot, lettre_cherchee)`

Code Python

```python
def compte(mot, lettre_cherchee):
    ''' Renvoie le nombre de fois que lettre_cherchee est dans mot'''




    #
```

Exercice 9 : Ecrire la fonction `compte2(mot, lettre_cherchee)` qui itère sur les éléments

Code Python

```python
def compte2(mot, lettre_cherchee):
    ''' Renvoie le nombre de fois que lettre_cherchee est dans mot'''




#
```

II. La boucle WHILE

Les boucles WHILE

La boucle `WHILE` est une boucle non bornée. On ne sait à l'avance combien de fois on va la faire.

```
# En pseudo code
instruction1
tant que condition1 vraie:
    instruction2
instruction3
```

- instruction1 est effectuée
- tant que la condition1 est vraie on effectue l'instruction 2
- quand la condition1 est fausse, on sort de la boucle ;
- on effectue l'instruction3

Prototypage de la boucle WHILE en python

`condition1` est un booléen qui est évalué à chaque tour de boucle. Tant que `condition1` `True`, on effectue les instructions de la boucle. Lorsque `condition1` devient `False`, on sort de la boucle.

Code Python

```python
while condition1:
    instruction
```

Exercice 10 : Dire ce qu'affichent les programmes suivants et expliquer pourquoi

Code Python

```python
n = 0
u = 1
while u < 1000:
    n = n + 1
    u = u * 2
print(n)
print(u)
```

Code Python

```python
mot = 'abracadabra'
c = 0
trouve = False
while c < len(mot) and not(trouve):
    trouve = mot[c] == 'r'
    c = c + 1
print(trouve)
```

Exercice 11 : Boucle FOR

1. Écrire une fonction `somme(n)` qui renvoie la somme des entiers de 1 à n inclus. Par exemple, `somme(5)` doit renvoyer $1+2+3+4+5=15$.

Code Python

```python
def somme(n):
    ''' Renvoie la somme des entiers de 1 à n inclus'''



#
```

2. Écrire une fonction `factorielle(n)` qui renvoie la factorielle de n. Par exemple, `factorielle(5)` doit renvoyer $1\times2\times3\times4\times5=120$.

Code Python

```python
def factorielle(n):
    ''' Renvoie la factorielle de n'''



#
```

3. Ecrire une fonction `remplace_lettre(mot, lettre_a_remplacer, nouvelle_lettre)` qui remplace toutes les occurrences de `lettre_a_remplacer` par `nouvelle_lettre` dans `mot`. Par exemple, `remplace_lettre('abracadabra', 'a', 'o')` doit renvoyer `'obrocodobro'`.

Code Python
```python
def remplace_lettre(mot, lettre_a_remplacer, nouvelle_lettre):
    ''' Renvoie mot dans lequel toutes les occurrences de
    lettre_a_remplacer ont été remplacées par nouvelle_lettre'''




#
```

4. Ecrire une fonction `supprime_lettre(mot, lettre_a_supprimer)` qui supprime toutes les occurrences de `lettre_a_supprimer` dans `mot`. Par exemple, `supprime_lettre('abracadabra', 'a')` doit renvoyer `'brcdbr'`.

Code Python
```python
def supprime_lettre(mot, lettre_a_supprimer):
    ''' Renvoie mot dans lequel toutes les occurrences de
    lettre_a_supprimer ont été supprimées'''




#
```

Exercice 12 : Boucle While

1. On dispose d'une feuille de papier d'épaisseur 0,1 mm. Combien de fois doit-on la plier au minimum pour que l'épaisseur dépasse la hauteur de la tour Eiffel 324 m. Écrire une fonction `nb_plis()` en Python pour résoudre ce problème.

2. Inès veut construire une pyramide à base carrée comme sur la photo. La pyramide sur la photo a 7 étages.
Inès a n billes. Combien d'étages au maximum aura sa pyramide ? Écrire une fonction `nb_etages(n)` en Python pour répondre au problème.

Code Python
```python
assert nb_etages(1000) == 14
```

3. L'ordinateur tire un nombre entier au hasard entre 0 et 100. L'utilisateur doit le trouver et pour cela propose des valeurs. L'ordinateur indique pour chaque valeur proposée si la valeur est trop petite, trop grande ou s'il a trouvé.
Écrire une fonction `jeu()` en Python pour jouer à ce jeu. En combien de coups est-on sûr de trouver ?

Méthode 1 : La bibliothèque `random`

Code Python
```python
import random
# pour tirer un nombre entier au hasard entre 0 et 100
nombre_aleatoire = random.randint(0, 100)
```

4. En mathématiques, la suite de Fibonacci est une suite d'entiers dans laquelle chaque terme est la somme des deux termes qui le précèdent. Elle commence généralement par les termes 0 et 1 et ses premiers termes sont 0, 1, 1, 2, 3, 5, 8, 13, 21, 34, 55, 89, etc.

   - Ecrire une fonction `fibonacci(n)` qui renvoie le terme de rang n de la suite de Fibonnaci. (le premier terme est de rang 0)

     Code Python
     ```python
     assert fibonacci(0) == 0
     assert fibonacci(1) == 1
     assert fibonacci(2) == 1
     assert fibonacci(9) == 34
     ```

   - Ecrire une fonction `rang_fibonacci()` qui renvoie le rang du premier terme supérieur ou égal à 10000.

     Code Python
     ```python
     assert rang_fibonacci() == 21
     ```

5. L'utilisateur donne un entier positif et le programme annonce combien de fois de suite cet entier est divisible par 2. Ecrire la fonction `divise_par_2(n)` qui renvoie ce nombre de fois. Par exemple, `divise_par_2(48)` doit renvoyer 4 car $48 = 2^4 \times 3$ et 4 est le plus grand entier tel que 48 est divisible par 2^4.

   Code Python
   ```python
   assert divise_par_2(48) == 4
   assert divise_par_2(7) == 0
   ```

6. Écrire une fonction `tirage()` qui fait des tirages successifs d'entiers 0 ou 1 jusqu'à tirer 3 fois 1 consécutivement et renvoie le nombre de tirages qui ont été nécessaires.

   Méthode 2 : La bibliothèque `random`

   Code Python
   ```python
   import random
   # pour tirer un nombre entier au hasard entre 0 et 1
   nombre_aleatoire = random.randint(0, 1)
   ```

7. Ecrire une fonction python qui renvoie l'entier p tel que $2^p \leqslant n \leqslant 2^{p+1}$.